BTC DATA PIPELINE

Ciclo de Vida de Engenharia de Dados – Parte 1

https://github.com/joaobizzo/btc-data-pipeline

João Bizzo, Pietra Paz

CONTEXTO E PROBLEMA

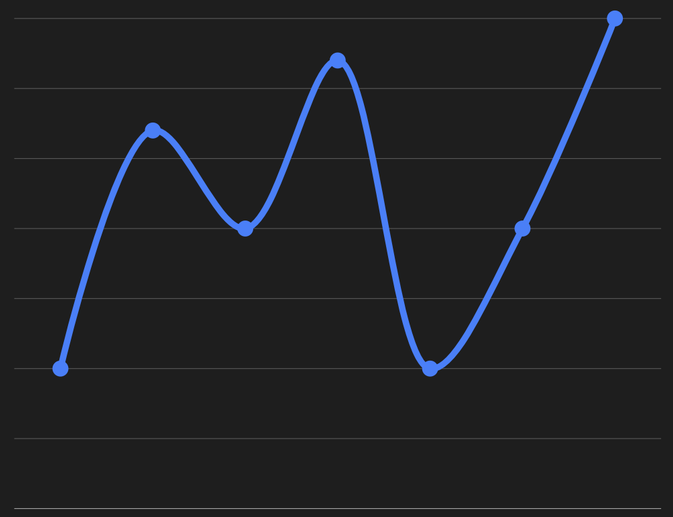
O CENÁRIO

- **Projeto:** Btc Data Pipeline
- **Setor:** Financeiro / Criptoativos
- Mercado de alta volatilidade 24/7
- Necessidade de decisões rápidas

O DESAFIO

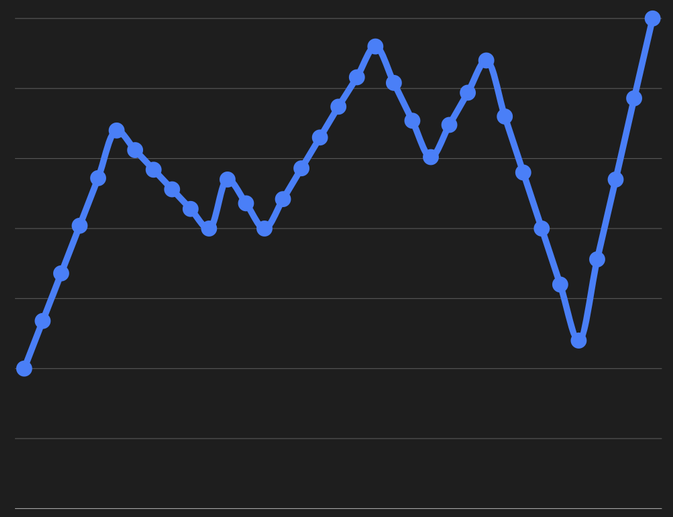
- Equilibrar granularidade vs. custo
- Histórico longo vs. Resolução alta
- Recuperação automática de falhas
- **Stakeholders:** Investidores e Devs

BAIXA GRANULAÇÂO



X

ALTA GRANULAÇÂO



DADOS UTILIZADOS

Fonte: API CoinGecko (JSON)

OPERACIONAIS (BATCH)

- Endpoint: **/market_chart**
- Séries temporais históricas completas
- Uso: Bootstrap e Backfill automático
- Maior volume de dados por requisição

STREAMING (SIMULADO)

- Endpoint: **/simple/price**
- Preços atuais de baixa latência
- Uso: Monitoramento recente (Speed Layer)
- Payload reduzido para coletas frequentes

DOMÍNIOS E SERVIÇOS

DOMÍNIO	SERVIÇOS PRINCIPAIS
INGESTÃO DE MERCADO	■ Coleta BTC/USD ■ Coleta Taxa USD/BRL
ANALYTICS E TRANSFORMAÇÃO	■ Armazenamento ■ Controle de Granularidade ■ Recuperação (Backfill)
CONSUMO	■ Metabase ■ API de Disponibilização

ARQUITETURA ESCOLHIDA

MODELO: MEDALHÃO

Combina a estrutura de camadas **Bronze/Silver** com a separação entre **Speed Layer** (coleta contínua) e **Batch Layer** (reconstrução histórica). Garante resiliência e recuperação automática de dados.

JUSTIFICATIVA

- Separação de responsabilidades e baixo acoplamento
- Facilidade de reprocessamento e resiliência
- Equilíbrio entre flexibilidade e consumo analítico

TRADE-OFF PRINCIPAL

Dependência da precisão do **provedor único** (CoinGecko) para manter a simplicidade e o baixo custo operacional do pipeline.

FLUXO DE DADOS PONTA A PONTA

INGESTÃO DUAL

- Speed: /simple/price (baixa latência)
- Batch: /market_chart (histórico)
- Resiliência contra downtime da API

PROCESSAMENTO SILVER

- Conversão de Unix Timestamp para ISO-8601
- Cálculo de câmbio implícito USD/BRL
- Normalização para consumo analítico

TECNOLOGIAS POR ETAPA

ETAPA	TECNOLOGIA	POR QUÊ?
INGESTÃO	**Python + Requests**	Simples, leve e flexível para integração com APIs REST.
ARMAZENAMENTO	**PostgreSQL**	Robusto para séries temporais e excelente suporte local.
PROCESSAMENTO	**Pandas / numpy**	Transformações e agregações eficientes com SQL ou Python.
ORQUESTRAÇÃO	**Airflow**	Gestão profissional de agendamento, dependências e backfill.
CONSUMO	**Metabase**	Criação rápida de dashboards intuitivos e conexão nativa.

SEGURANÇA E GOVERNANÇA

SEGURANÇA

- Proteção de credenciais via variáveis de ambiente
- Acesso restrito aos endpoints da API
- Criptografia de dados em repouso no Postgres

GOVERNANÇA

- Camada Bronze imutável para auditoria
- Linhagem de dados clara (Medalhão)
- Padronização de esquemas na camada Silver

MONITORAMENTO

- Logs detalhados de execução no Airflow
- Alertas automáticos em caso de falha na ingestão
- Health checks periódicos da base de dados

RISCOS E PRÓXIMOS PASSOS

PRINCIPAIS RISCOS

- **Rate Limiting:** Bloqueios por excesso de requisições no endpoint de Speed.
- **Dependência de API:** Mudanças estruturais no payload do market_chart.
- **Conectividade:** Falhas de rede impactando a sincronização em tempo real.

PARTE 2: EVOLUÇÃO

- Automação completa do **Backfill** via market_chart.
- Implementação da **Camada Gold** para análises de volatilidade.
- Iniciar a integração com o backend existente
- Otimização de índices para séries temporais.